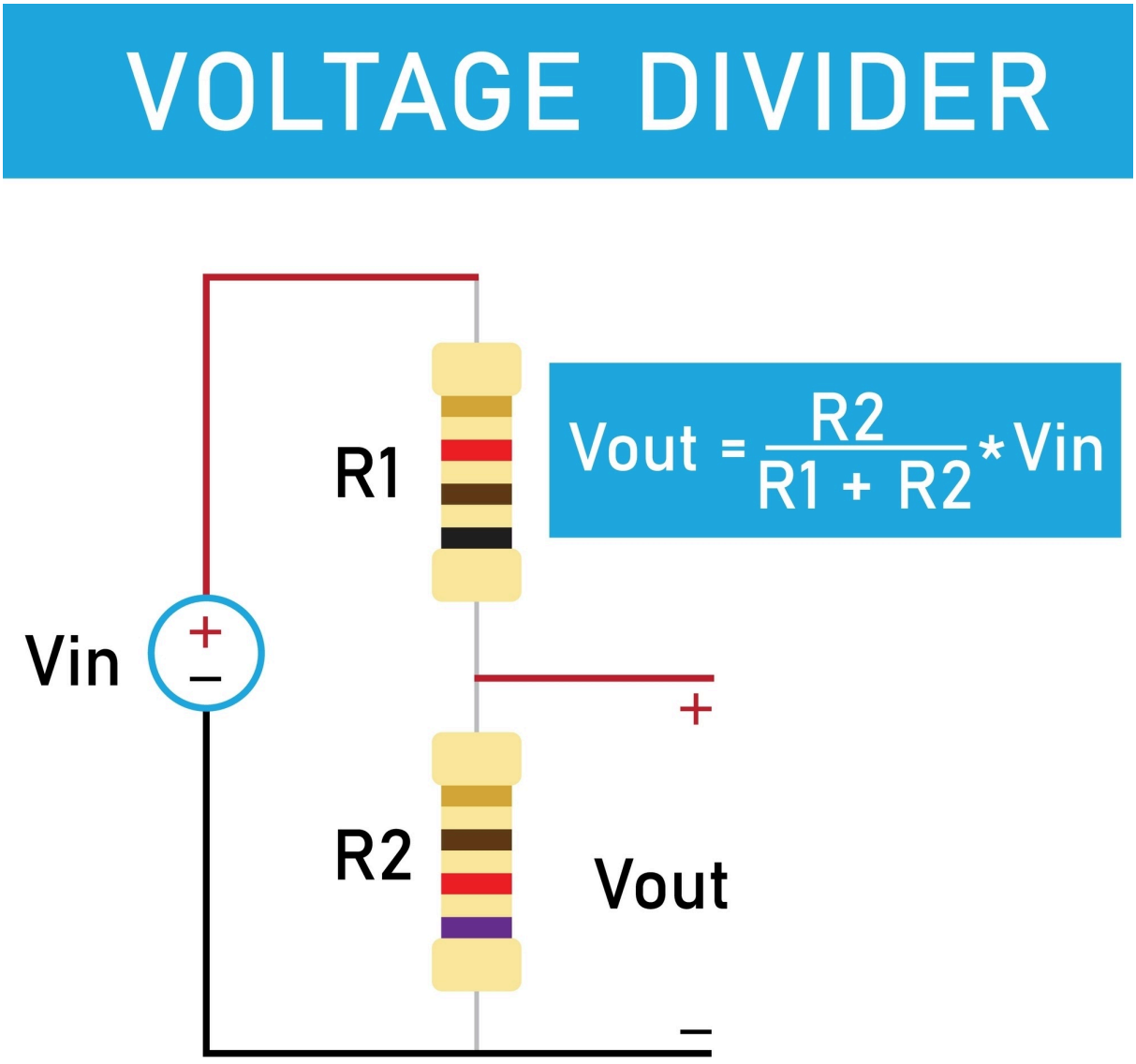
Project:1 Real-Time Environmental Data Logger and Visualizer

Goal: *Build a physical circuit to measure ambient light levels, send the raw analog data to machine via serial communication, and write a Python script to filter, process, and plot this real-world data stream in real-time.*

Notes:

Concept 1: The Voltage Divider

Your microcontroller possesses an ADC (Analog-to-Digital Converter). An ADC can only measure **voltage**. It cannot measure resistance. However, our sensor—the LDR—changes its *resistance* based on how much light hits it. We have to map a change in resistance to a change in voltage so the ADC can read it. We achieve this using a Voltage Divider circuit.



A voltage divider consists of two resistors, R_1 and R_2, connected in series across a power supply. The supply voltage is V_{in}. We measure our output voltage, V_{out}, across R_2.The governing equation derived from Ohm's Law is:

$$V_{\text{out}} = V_{\text{in}} \cdot \frac{R_2}{R_1 + R_2}$$

Example Calculation: Voltage Divider with an LDR

Given

- Microcontroller supply voltage:

$$V_{\text{in}} = 5\text{V}$$

- Fixed resistor:

$$R_1 = 10\,\text{k}\Omega = 10{,}000\,\Omega$$

- LDR resistance under bright room light:

$$R_2 = 2\,\text{k}\Omega = 2{,}000\,\Omega$$

Voltage Divider Equation

$$V_{\text{out}} = V_{\text{in}} \cdot \frac{R_2}{R_1 + R_2}$$

Step 1: Substitute the Values

$$V_{\text{out}} = 5 \cdot \frac{2000}{10000 + 2000}$$

Step 2: Simplify the Denominator

$$V_{\text{out}} = 5 \cdot \frac{2000}{12000}$$

Step 3: Perform the Division

$$V_{\text{out}} = 5 \cdot 0.1667$$

Step 4: Calculate the Output Voltage

$$V_{\text{out}} = 0.833\text{V}$$

> **Interpretation**
>
> The voltage divider produces an output voltage of approximately **0.833 V** when the LDR is exposed to bright room light.

What Happens Next?

1. The microcontroller's Analog-to-Digital Converter (ADC) measures the analog voltage:

$$0.833\text{V}$$

1. The ADC converts this analog voltage into a digital integer.
2. The integer is transmitted over the serial connection.
3. Your Python script receives the integer, processes it, and uses it for visualization or further analysis.

Concept 2: Analog-to-Digital Conversion (ADC) Quantization

Your microcontroller's processor is entirely digital; it operates on binary logic. It has absolutely no concept of "4.166V." To get this real-world voltage into your eventual Python script, the microcontroller must use its Analog-to-Digital Converter (ADC).

ADC Conversion Example

Step 1: Calculate the Total Number of ADC Bins (Resolution)

The number of discrete output values produced by an ADC is given by:

$$\text{Total Bins} = 2^n$$

where:

- n = number of ADC bits

For a **10-bit ADC**:

$$\text{Total Bins} = 2^{10} = 1024$$

> **Interpretation**
>
> The ADC can output **1024 discrete values**, ranging from:
>
> $$0 \text{ to } 1023$$

Step 2: Calculate the Voltage Resolution (Step Size)

Each increment of the ADC corresponds to a small change in voltage.

The voltage represented by one ADC count is:

$$V_{\text{step}} = \frac{V_{\text{ref}}}{\text{Total Bins}}$$

For an Arduino using a **5 V reference**:

$$V_{\text{ref}} = 5\text{V}$$

Substitute the values:

$$V_{\text{step}} = \frac{5}{1024}$$

$$V_{\text{step}} = 0.0048828\text{V}$$

$$V_{\text{step}} \approx 4.88\text{mV}$$

> **Interpretation**
>
> Every increase of **1 ADC count** represents approximately **4.88 mV**.

Step 3: Convert the Analog Voltage into an ADC Reading

From the previous voltage-divider calculation:

$$V_{\text{out}} = 0.833\text{V}$$

The ADC value is calculated as:

$$\text{ADC Value} = \frac{V_{\text{out}}}{V_{\text{step}}}$$

Substitute the known values:

$$\text{ADC Value} = \frac{0.833}{0.0048828}$$

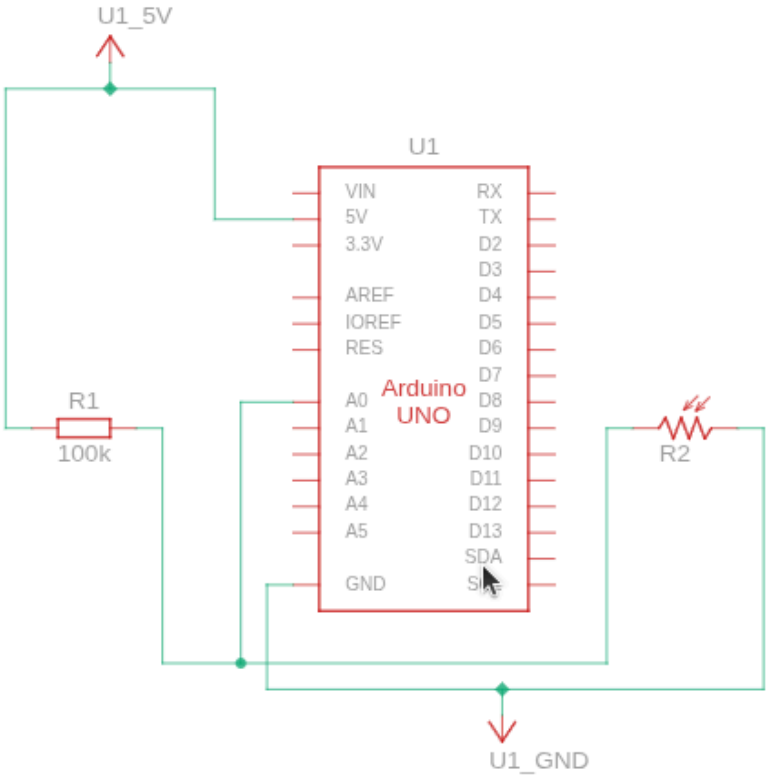
$$\text{ADC Value} = 170.59$$

Step 4: Final ADC Reading

Since the ADC produces an integer output, the fractional value is converted to the nearest integer:

$$\boxed{\text{ADC Value} = 171}$$

> **Interpretation**
>
> When the voltage at the analog input is approximately **0.833 V**, the Arduino's 10-bit ADC converts it into the digital value:
>
> $$\boxed{171}$$
>
> This integer is transmitted through the serial port and becomes the value that your Python program receives and processes.



Concept 3: Serial Communication and The Baud Rate Bottleneck

1. The Concept

Your Arduino now has an integer (853) sitting in its local memory. Your PC, where your Python ML environment lives, has no access to it. We must physically transmit this data over the USB cable using Serial Communication.

The microcontroller sends data by rapidly pulsing a single transmit wire (TX) High (5V) and Low (0V) to represent binary 1s and 0s. Because there is no shared clock signal wire to synchronize the two devices, both the Arduino and your Python script must strictly agree in advance on the exact speed of

these pulses. This speed is the **Baud Rate**, measured in bits per second (bps).

Serial Transmission Time Calculation

Step 1: Determine the Number of Characters Sent

When you execute:

```
Serial.println(853);
```

the Arduino does **not** transmit the integer directly. Instead, it sends the **ASCII representation** of the number.

Characters transmitted:

- `'8'` → 1 byte
- `'5'` → 1 byte
- `'3'` → 1 byte

`Serial.println()` also appends:

- Carriage Return (`\r`) → 1 byte
- Newline (`\n`) → 1 byte

Therefore,

$$\text{Total Characters} = 5$$

Since each ASCII character occupies **1 byte**,

$$\text{Total Bytes} = 5$$

Step 2: Calculate the Total Number of Bits

Using the standard **8-N-1** serial protocol, each byte consists of:

- 1 Start bit
- 8 Data bits
- 1 Stop bit

Total:

$$10 \text{ bits per byte}$$

Therefore,

$$\text{Total Bits} = 5 \times 10 = 50 \text{ bits}$$

Step 3: Calculate the Transmission Time

Assume the serial baud rate is:

$$9600 \text{ bps}$$

The transmission time is calculated using:

$$T = \frac{\text{Total Bits}}{\text{Baud Rate}}$$

Substitute the values:

$$T = \frac{50}{9600}$$

$$T = 0.005208 \text{ seconds}$$

Convert to milliseconds:

$$T = 5.208 \text{ ms}$$

Rounded:

$$\boxed{T \approx 5.2 \text{ ms}}$$

Step 4: Maximum Possible Transmission Rate

The maximum number of readings that can be transmitted per second is approximately:

$$\text{Readings per Second} = \frac{1}{0.005208}$$

$$= 192$$

Therefore,

$$\boxed{\text{Maximum Transmission Rate} \approx 192 \text{ readings/second}}$$

> **Interpretation**
>
> At a baud rate of **9600 bps**, transmitting the value `853` using `Serial.println()` requires approximately **5.2 ms**. Since each transmission occupies the communication channel during that time, the system is physically limited to roughly **192 readings per second**, assuming no additional processing or delays.

Concept 4: Buffer Accumulation and Processing Latency

In your AI/ML coursework, when you load a CSV dataset into Pandas, the entire dataset is instantly available in RAM. Serial data does not work like this. It is a continuous, asynchronous stream.

Your operating system (Windows/Linux) catches these incoming bytes and holds them in a **Serial Buffer** (a temporary queue in RAM) until your Python script asks for them. If your Python script takes too long to process and plot the data, the Arduino will keep shoving data into the buffer faster than Python can remove it. The buffer will accumulate until it overflows, causing a fatal system crash or catastrophic data corruption.

Buffer Overflow Time Calculation

Step 1: Define the Data Rates

Let:

- R_{in} = Hardware data generation rate (readings per second)
- R_{out} = Software processing rate (readings per second)

The rate at which data accumulates in the serial buffer is:

$$A = R_{\text{in}} - R_{\text{out}}$$

where:

- A = Accumulation Rate (readings per second)

Step 2: Hardware Data Rate

Suppose the Arduino transmits data at:

$$R_{\text{in}} = 200 \text{ readings/second}$$

Step 3: Software Processing Rate

Assume the Python visualization uses a computationally expensive Matplotlib operation that requires:

$$20 \text{ ms}$$

to process one frame.

Convert milliseconds to seconds:

$$20 \text{ ms} = 0.020 \text{ s}$$

The processing rate is the reciprocal of the processing time:

$$R_{\text{out}} = \frac{1}{0.020}$$

$$R_{\text{out}} = 50 \text{ readings/second}$$

Step 4: Calculate the Accumulation Rate

Substitute the values into the accumulation equation:

$$A = R_{\text{in}} - R_{\text{out}}$$

$$A = 200 - 50$$

$$A = 150 \text{ readings/second}$$

Step 5: Buffer Capacity

Suppose the operating system's serial buffer can store exactly:

$$B = 3000 \text{ readings}$$

Step 6: Calculate the Time Until Buffer Overflow

The time until the buffer becomes full is:

$$T = \frac{B}{A}$$

Substitute the known values:

$$T = \frac{3000}{150}$$

$$T = 20 \text{ seconds}$$

Final Answer

$$\boxed{T = 20 \text{ seconds}}$$

> **Interpretation**

> The Arduino generates data at **200 readings per second**, while the Python application can process only **50 readings per second**. As a result, **150 unread readings accumulate in the serial buffer every second**. With a buffer capacity of **3000 readings**, the buffer becomes completely full after **20 seconds**.
>
> Once the buffer reaches capacity, new incoming data can no longer be stored. Depending on the operating system, serial driver, and application behavior, this may result in dropped readings, increased latency, communication stalls, or buffer overrun conditions. A well-designed real-time system prevents this by ensuring that the processing rate matches or exceeds the incoming data rate, or by intentionally discarding stale data when appropriate.

Concept 5: Sensor Noise and Exponential Smoothing

Physical sensors are never mathematically perfect. Electromagnetic interference (EMI) from the USB cable, thermal noise in the resistor, and microscopic fluctuations in the ADC mean your signal will jitter randomly even if the physical lighting in the room is completely static.

Exponential Moving Average (EMA) Example

Exponential Moving Average Formula

The Exponential Moving Average (EMA) is calculated using the following equation:

$$S_t = \alpha \cdot Y_t + (1 - \alpha) \cdot S_{t-1}$$

where:

- S_t = New filtered value (the value plotted on the graph)
- Y_t = Current raw sensor reading received from the serial buffer

- S_{t-1} = Previous filtered value
- α = Smoothing factor, where:

$$0 < \alpha \leq 1$$

Given

Assume the following conditions:

- Previous filtered value:

$$S_{t-1} = 850$$

- Current raw sensor reading:

$$Y_t = 880$$

- Smoothing factor:

$$\alpha = 0.2$$

Step 1: Substitute the Values

$$S_t = 0.2 \times 880 + (1 - 0.2) \times 850$$

Step 2: Simplify the Equation

$$S_t = 176 + (0.8 \times 850)$$

Step 3: Perform the Multiplication

$$S_t = 176 + 680$$

Step 4: Calculate the Filtered Value

$$S_t = 856$$

Final Answer

$$\boxed{S_t = 856}$$

Interpretation

The raw sensor reading suddenly increased from:

$$850 \rightarrow 880$$

This is a spike of:

$$880 - 850 = 30$$

However, the EMA output changed from:

$$850 \rightarrow 856$$

which is only:

$$856 - 850 = 6$$

Instead of reacting to the full **30-unit spike**, the filter adjusts by only **6 units**, significantly reducing the effect of transient electrical noise.

This demonstrates why the Exponential Moving Average is widely used in embedded systems, robotics, and real-time sensor processing. By blending the current measurement with the previous filtered value, the EMA produces a smoother and more stable signal while still adapting to genuine changes in the sensor readings.

Concept 6: GUI Thread Blocking in Real-Time Systems

Why `matplotlib.pyplot.show()` Cannot Be Used for Real-Time Data

Blocking vs. Non-Blocking Visualization

In many data science projects, a graph is displayed using:

```
plt.show()
```

This function is **blocking**, meaning the Python interpreter pauses execution until the graph window is manually closed.

As a result, any code placed after `plt.show()` does not execute until the window is closed.

Why This Fails in a Real-Time System

Suppose `plt.show()` is placed inside a continuous serial-reading loop.

```
while True:
    value = read_serial()
    plt.plot(value)
    plt.show()
```

During the **first iteration**, the program pauses to display the graph.

While the Python program is waiting:

- The Arduino continues transmitting sensor readings.
- Data continues arriving over the USB serial connection.
- The operating system stores incoming data in the serial buffer.

Eventually, the serial buffer becomes full.

From our previous calculation, if:

- Hardware sends:

$$R_{\text{in}} = 200 \text{ readings/second}$$

- Software processes only:

$$R_{\text{out}} = 50 \text{ readings/second}$$

Then:

$$A = R_{\text{in}} - R_{\text{out}}$$

$$A = 200 - 50 = 150 \text{ readings/second}$$

With a buffer capacity of:

$$B = 3000 \text{ readings}$$

the overflow time is:

$$T = \frac{B}{A}$$

$$T = \frac{3000}{150}$$

$$T = 20 \text{ seconds}$$

Therefore, the serial buffer overflows after:

$$\boxed{20 \text{ seconds}}$$

The Solution: Non-Blocking Animation

Instead of repeatedly calling `plt.show()`, Matplotlib provides:

```
matplotlib.animation.FuncAnimation
```

`FuncAnimation` keeps the Python program running while periodically updating the graph.

This separates the application into two independent tasks:

- **Backend:** Continuously reads sensor data from the serial port.
- **Frontend:** Periodically redraws the graph using the latest available data.

Because these tasks operate independently, data collection continues even while the graph is being refreshed.

Animation Update Interval

`FuncAnimation` requires an update interval measured in milliseconds.

The relationship between frame rate and update interval is:

$$I = \frac{1000 \text{ milliseconds}}{\text{FPS}}$$

where:

- I = Animation interval (milliseconds)
- FPS = Desired frames per second

Example: 10 FPS

Suppose we want the graph to refresh at:

$$10 \text{ FPS}$$

Substitute into the equation:

$$I = \frac{1000}{10}$$

$$I = 100 \text{ ms}$$

Therefore,

$$\boxed{I = 100 \text{ ms}}$$

What Happens During Those 100 ms?

Assume the Arduino transmits data at:

$$100 \text{ readings/second}$$

Since:

$$100 \text{ ms} = 0.1 \text{ seconds}$$

the number of readings collected between screen updates is:

$$100 \times 0.1 = 10$$

Thus, while the display remains unchanged, the backend quietly accumulates approximately:

$$\boxed{10 \text{ new sensor readings}}$$

When the animation callback executes after **100 ms**, it redraws the graph using all newly collected data points in a single update.

Interpretation

Using `FuncAnimation` allows the serial communication loop to run continuously while the graphical interface refreshes at a controlled frame rate. Instead of blocking the program after every reading, sensor data is collected in the background and displayed periodically. This design keeps the visualization responsive, minimizes the risk of serial buffer overflow, and enables reliable real-time monitoring of continuously changing physical signals.

Real-Time Serial Visualization with Matplotlib

This program continuously receives sensor readings from an Arduino through a serial connection and displays them as a

scrolling real-time graph. The serial communication runs continuously in the background, while the graphical interface refreshes periodically using `matplotlib.animation.FuncAnimation`.

Step 1: Import the Required Libraries

```
import serial
import time
import collections
import matplotlib.pyplot as plt
import matplotlib.animation as animation
```

Each library has a specific responsibility.

- `serial` → Communicates with the Arduino over USB.
- `time` → Provides timing functions such as delays.
- `collections` → Provides the `deque` data structure for efficient rolling windows.
- `matplotlib.pyplot` → Creates the graph and figure.
- `matplotlib.animation` → Updates the graph continuously without blocking the program.

Step 2: Configure the Serial Connection

```
ser = serial.Serial('COM3', 38400, timeout=1)
time.sleep(2)
```

A serial connection is established using:

- Port:

```
COM3
```

(or `/dev/ttyACM0` on Linux)

- Baud Rate:

$$38400 \text{ bits/second}$$

The program then pauses for:

$$2 \text{ seconds}$$

This delay allows the Arduino to automatically reset and begin transmitting data before Python starts reading from the serial port.

Step 3: Create a Rolling Data Window

```
data_window = collections.deque([0]*100, maxlen=100)
```

A **deque** (double-ended queue) is created.

Initial contents:

$$[0, 0, 0, \ldots, 0]$$

containing:

$$100$$

values.

The maximum size is fixed at:

$$100$$

Whenever a new reading is added:

- the newest value is appended to the right.
- the oldest value is automatically discarded.

Therefore,

$$\boxed{\text{Window Size} = 100}$$

at all times.

This creates a scrolling graph that always displays the most recent **100 sensor readings**.

Step 4: Create the Figure

```
fig, ax = plt.subplots()
```

This creates:

- one Figure
- one Axes

The Figure represents the window, while the Axes represent the plotting area.

Step 5: Create the Plot

```
line, = ax.plot(data_window)
```

A line object is created using the initial contents of `data_window`.

Initially, the graph displays a flat horizontal line because every value is zero.

Step 6: Configure the Axes

```
ax.set_ylim(0, 1023)
```

The vertical axis is limited to:

$$0 \leq y \leq 1023$$

These limits exactly match the output range of a **10-bit ADC**:

$$2^{10} = 1024$$

possible values

$$0 \rightarrow 1023$$

The axis labels are then assigned:

```
ax.set_xlabel("Samples")
ax.set_ylabel("ADC Value (0-1023)")
```

The horizontal axis represents the most recent samples.

The vertical axis represents the measured ADC values.

Step 7: Define the Update Function

```
def update(frame):
```

This function is called automatically by `FuncAnimation` every animation interval.

The parameter

```
frame
```

is supplied by Matplotlib but is not used in this program.

Step 8: Empty the Serial Buffer

```
while ser.in_waiting > 0:
```

The program checks whether unread bytes are waiting in the serial buffer.

If multiple sensor readings have accumulated since the previous screen refresh, the loop processes **every available**

reading before updating the graph.

This prevents serial buffer accumulation.

Step 9: Read One Sensor Reading

```
raw_data = ser.readline().decode('utf-8').strip()
```

This line performs three operations:

1.

```
readline()
```

Reads one complete line from the serial port.

1.

```
decode('utf-8')
```

Converts the incoming bytes into a Python string.

1.

```
strip()
```

Removes whitespace characters such as:

- `\n`
- `\r`

For example,

Arduino sends:

```
853\r\n
```

Python receives:

```
"853"
```

Step 10: Validate the Data

```
if raw_data.isdigit():
```

The reading is processed only if it contains digits.

Valid examples:

```
853
```

```
421
```

```
1023
```

Invalid examples:

```
85.3
```

```
abc
```

```
-12
```

This prevents conversion errors.

Step 11: Store the New Reading

```
data_window.append(int(raw_data))
```

The string is converted into an integer.

The new reading is appended to the deque.

If the deque already contains:

$$100$$

values,

the oldest value is automatically removed.

Therefore,

$$\boxed{\text{Length of the deque always remains } 100}$$

Step 12: Update the Graph

```
line.set_ydata(data_window)
```

The graph is **not recreated**.

Instead, the existing line object receives the newest sensor values.

Only the displayed data changes.

This is significantly more efficient than generating a new graph during every update.

Step 13: Return the Updated Object

```
return line,
```

The trailing comma creates a one-element tuple.

When

```
blit=True
```

is enabled, Matplotlib redraws only the returned graphical objects.

Instead of repainting the entire window, only the modified line is refreshed.

This substantially improves animation performance.

Step 14: Create the Animation

```
ani = animation.FuncAnimation(
    fig,
    update,
    interval=42,
    blit=True
)
```

`FuncAnimation` repeatedly calls the `update()` function.

Parameters:

- **Figure**

```
fig
```

The window that will be animated.

- **Update Function**

```
update
```

Executed once every animation interval.

- **Animation Interval**

```
interval=42
```

The graph refreshes every:

$$42 \text{ ms}$$

The corresponding frame rate is:

$$\text{FPS} = \frac{1000}{42}$$

$$\text{FPS} \approx 23.81$$

which is the closest integer interval to the cinematic standard of:

$$24\ \text{FPS}$$

- **Blitting**

```
blit=True
```

Only modified graphical objects are redrawn, reducing CPU usage and improving responsiveness.

Step 15: Display the Window

```
plt.show()
```

Unlike a static plot, the window now enters Matplotlib's event loop.

While the window remains open:

- the serial-reading callback continues collecting sensor data,
- the GUI refreshes every **42 ms**,
- the graph continuously scrolls as new readings arrive.

The backend data acquisition and frontend visualization operate concurrently, allowing real-time monitoring of the sensor stream without blocking serial communication.

Overall Program Flow

The execution sequence of the program is:

1. Import the required libraries.
2. Establish a serial connection with the Arduino.
3. Create a fixed-size rolling buffer containing the latest 100 readings.
4. Construct the plotting window and configure its axes.
5. Register the `update()` callback with `FuncAnimation`.
6. Continuously receive sensor readings from the serial port.
7. Append each valid reading to the rolling buffer.
8. Refresh the graph every **42 ms** using the latest available data.
9. Continue this process until the program is terminated.